

TREES

DAY 170
27/12/2025

Non-Recursive Postorder Traversal

In pre order and in order traversals, after popping the stack element, we do not need to visit the same vertex again.

But in the post order traversal, each node is visited twice. That means, after processing the left subtree we will visit the current node and after processing the right subtree we will visit the same current node. But we should be processing the node during the second visit. Here the problem is how to differentiate whether we are returning from the left subtree or the right subtree.

We use a previous variable to keep track of the earlier traversed node.

Let's assume "Current" is the current node that is on top of the stack. When previous is current's parent, we are traversing down the tree.

In this case, we try to traverse to current's left child if available (i.e., push left child to the stack). If it is not available, we look at "current's" right child. If both left & right child do not exist (i.e., current is a leaf node)

We print current's value and pop it off the stack

If prev is current's left child, we are traversing up the tree from the left. We look at current's right child. If it is available, then traverse down the right child (i.e., push right child to the stack); otherwise print current's value and pop it off the stack. If previous is current's right child, we are traversing up the tree from the right. In this case, we print current's value and pop it off the stack.

```
public static void postOrder (TreeNode root) {
```

```
    Stack<TreeNode> st = new Stack<>();
```

```
    TreeNode cur = root;
```

```
    TreeNode lastVisited = null;
```

```
    while (cur != null || !st.isEmpty()) {
```

```
        while (cur != null) {
```

```
            st.push(cur);
```

```
            cur = cur.left;
```

```
        }
```

```
        TreeNode peek = st.peek();
```

```
        if (peek.right != null && lastVisited != peek.right) {
```

```
            cur = peek.right;
```

```
        }
```

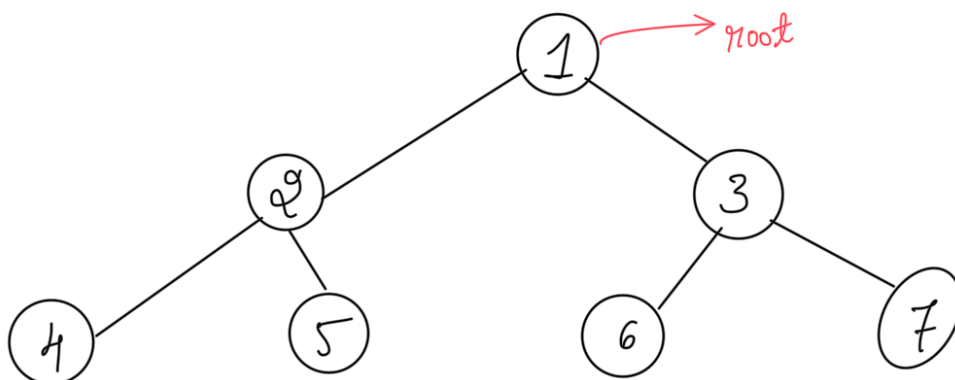
```
    } else {
```

```
        System.out.print(peek.data + " ");
```

```
        lastVisited = st.pop();
```

```
    }
```

```
}
```



Cur = 1
stack = []
last Visited = null

Step 2 peek = 4

4 has no right child
print 4
last Visited = 4
stack = [1, 2]

Step 4 no left

push 5
Cur = null
stack = [1, 2, 5]

Step 6 : peek = 2

right child = 5
last Visited == 5
→ print 2
last Visited = 2
stack = [1]

Step 8 no left

push 3 → push 6
Cur = null
stack = [1, 3, 6]

Step 1 no left

push 1 → push 2 → push 4
Cur = null
stack = [1, 2, 4]

Step 3 peek = 2

right child = 5
last Visited ≠ 5
move to right
Cur = 5

Step 5 peek = 5

no right child
print 5
last Visited = 5
stack = [1, 2]

Step 7 peek = 1

right child = 3
last Visited ≠ 3
move to right
Cur = 3

Step 9 peek = 6

no right child
print 6
last Visited = 6
stack = [1, 3]

Step 10 peek = 3

right child = 7
last Visited $\neq$ 7
cur = 7

Step 12 peek = 7
no right child
print 7
last Visited = 7
stack = [1, 3]

Step 14 peek = 1
Right child = 3
last Visited = 3
print 1
stack = []

Step 11 Go left

push 7
cur = null
stack = [1, 3, 7]

Step 13 peek = 3
right child = 7
print 3
last Visited = 3
stack = [1]

Final output of Traversal

4	5	2	6	7	3	1
---	---	---	---	---	---	---